

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  // Structure to represent a block
5  struct Block {
6      char data[256]; // Adjust the size as needed for your blocks
7      struct Block* next;
8  };
9
10 int main() {
11     struct Block* firstBlock = NULL; // Pointer to the first block in the linked list
12     struct Block* lastBlock = NULL; // Pointer to the last block in the linked list
13
14     int blockCount = 0; // Count of blocks in the linked list
15
16     int blockNumber;
17     char data[256];
18     char choice;
19
20     printf("Linked Allocation Simulation\n");
21
22     while (1) {
23         printf("Enter 'W' to write a block, 'R' to read a block, or 'Q' to quit: ");
24         scanf(" %c", &choice);
25
26         if (choice == 'Q' || choice == 'q') {
27             break;
28         }
29
30         if (choice == 'W' || choice == 'w') {
31             printf("Enter data for the block: ");
```

```
Linked Allocation Simulation
Enter 'W' to write a block, 'R' to read a block, or
'Q' to quit: W
Enter data for the block: Block_Alpha_Data_Section
Enter 'W' to write a block, 'R' to read a block, or
'Q' to quit: W
Enter data for the block: Block_Beta_Data_Section
Enter 'W' to write a block, 'R' to read a block, or
'Q' to quit: R
Enter the block number to read (1-2): 1
Block 1 Data: Block_Alpha_Data_Section
Enter 'W' to write a block, 'R' to read a block, or
'Q' to quit: R
Enter the block number to read (1-2): 2
Block 2 Data: Block_Beta_Data_Section
Enter 'W' to write a block, 'R' to read a block, or
'Q' to quit: Q
```

=== Code Execution Successful ===

```

32     scanf("%[^\n]", data);
33
34     // Create a new block
35     struct Block* newBlock = (struct Block*)malloc(sizeof(struct Block));
36     for (int i = 0; i < 256; i++) {
37         newBlock->data[i] = data[i];
38     }
39     newBlock->next = NULL;
40
41     if (blockCount == 0) {
42         // This is the first block
43         firstBlock = newBlock;
44         lastBlock = newBlock;
45     } else {
46         // Link the new block to the last block
47         lastBlock->next = newBlock;
48         lastBlock = newBlock;
49     }
50
51     blockCount++;
52 } else if (choice == 'R' || choice == 'r') {
53     printf("Enter the block number to read (1-%d): ", blockCount);
54     scanf("%d", &blockNumber);
55
56     if (blockNumber < 1 || blockNumber > blockCount) {
57         printf("Invalid block number. The valid range is 1-%d.\n", blockCount);
58     } else {
59         struct Block* currentBlock = firstBlock;
60         for (int i = 1; i < blockNumber; i++) {
61             currentBlock = currentBlock->next;
62         }
63
64         printf("Block %d Data: %s\n", blockNumber, currentBlock->data);
65     }

```

```
66     }
67 }
68
69 // Free the allocated memory for blocks before exiting
70 struct Block* currentBlock = firstBlock;
71 while (currentBlock != NULL) {
72     struct Block* nextBlock = currentBlock->next;
73     free(currentBlock);
74     currentBlock = nextBlock;
75 }
76
77 return 0;
78 }
```